

Technical Report: Vision-Language Model Implementation

1 Problem

The goal of this project was to build a simple Vision-Language Model (VLM) from scratch. The model needs to take an input image containing basic geometric shapes and predict a text description of the objects' sizes, colors, shapes, and spatial relationships. My main learning objective was to understand how vision encoders and language decoders work together at a low level, specifically how to project visual features so a transformer decoder can attend to them without relying on pre-built libraries.

2 Architecture

The architecture follows a standard, modular encoder-decoder design to keep things simple and easy to test.

Tokenizer: Converts text into tokens using a fixed vocabulary of 27 tokens for attributes (sizes like *small*; colors like *red*; shapes like *circle*; relations like *leftof*) and control tokens.

Vision Encoder: A simple convolutional neural network (CNN) that takes an image and outputs a flattened 64x64 spatial feature grid.

Adapter: A linear layer that changes the CNN's output channels to match the decoder's hidden dimension size.

Decoder: A custom-written transformer decoder using Multi-Head Attention (MHA), a Multi-Layer Perceptron (MLP), and Layer Normalization.

VLM Wrapper: A single module that combines the CNN, adapter, and decoder.

3 Implementation

A few tricky implementation details were needed to get the model training correctly:

- **Causal Masking:** In the attention module, I had to implement a causal look-ahead mask for the self-attention in the decoder. This stops the model from "cheating" by looking at future tokens when trying to predict the current one.
- **Loss Masking:** During training, sequences are padded to the same length. I configured the Cross-Entropy loss to ignore these padding tokens so the model isn't penalized for them.
- **Correctness Check:** Before running the full training, I did a sanity check by trying to overfit a single batch of data (Seed 42). Over 99 epochs, the loss successfully went down from 3.437907 to 0.007649, which proved the math and gradients were working correctly.

4 Training Setup

Parameter	Value
Optimizer	AdamW
Learning Rate	3e-4
Batch Size	32
Epochs	14 (Saved best checkpoint based on validation loss)
Local Hardware	AMD Ryzen 5 7520U CPU with Radeon Graphics, Windows 11
Colab Hardware	NVIDIA Tesla T4 GPU (CUDA 12.8), Linux x86_64
Software	Python 3.12/3.13, PyTorch 2.4.1+cpu / 2.11.0+cu128
Seeds	42

Table 1: Model Training and Hardware Setup

5 Experiments

I ran two main experiments to test the model's capabilities and my hardware.

Experiment 1: Generalization

Question: Can the model generalize to new combinations of shapes and colors it hasn't seen before?

Hypothesis: It will memorize the training combinations but fail on completely new ones.

Setup: I evaluated the best checkpoint on three different data splits: standard test, test_heldout (novel combinations), and a completely blind test set.

Experiment 2: Hardware Throughput Benchmark

Question: How does processing speed (images/s) change with batch size on a CPU versus a GPU?

Hypothesis: The GPU will get much faster with larger batch sizes, while the CPU will struggle.

Setup: I tested batch sizes of 1, 16, 64, and 256 on both my local CPU and a Google Colab Tesla T4 GPU.

6 Results

Hardware Throughput Results

Figure 1 (Reference) S1 Training Throughput benchmark directly comparing CPU (blue) and CUDA GPU (orange) across different batch sizes. Notice how GPU throughput scales massively as batch size increases, while CPU throughput peaks early and slightly declines.

Metric	Test Split	Test Held-Out Split	Blind Test Split
Exact Match	70.35%	0.10%	1.65%
Size Accuracy	84.14%	83.76%	33.28%
Color Accuracy	74.40%	76.44%	20.10%
Shape Accuracy	74.24%	26.14%	20.63%
Relation Accuracy	43.43%	47.37%	0.00%
Unparsed Predictions	1 / 2000	18 / 2000	0 / 2000

Table 2: Evaluation performance across different test splits.

Task Accuracy Results

7 Analysis

The throughput benchmark graph clearly shows that GPU throughput becomes dramatically higher as batch size increases, maxing out around batch 64 and holding steady through 256 . Meanwhile, the CPU throughput peaks around batch 16 to 64 and actually decreases slightly at batch 256 . This is because the GPU has thousands of cores to handle the parallel matrix math of large batches, while the CPU gets bogged down by memory bottlenecks.

For the model's accuracy, the evidence strongly supports my hypothesis about generalization. On the standard test set, the model did well, hitting a 70.35% Exact Match and learning size and color reliably . However, on the test_heldout split, the Exact Match crashed to 0.10% . Interestingly, size and color accuracy stayed high (around 83% and 76%), but shape accuracy dropped from 74% to 26% . This proves the model didn't actually learn what the shapes look like; it just memorized specific color-shape pairs from the training set. Spatial relations ("above", "left of") were also very weak across all tests, barely breaking 47% .

8 Limitations

My biggest limitation was initially training on a CPU, which restricted me to only 14 epochs . I also didn't use any data augmentation (like randomly cropping or changing the colors of the images during training). If I had used data augmentation, the model might not have overfit to the color-shape combinations. The spatial relation accuracy is likely poor because my CNN doesn't have explicit 2D positional embeddings, so the attention mechanism doesn't know where things are on the grid. Next time, I would train fully on the Colab GPU for at least 50 epochs, add positional embeddings, and use heavy data augmentation.

9 References

1. Vaswani, A., et al. (2017). *Attention Is All You Need*. (Used to help write the attention formulas from scratch).
2. PyTorch Documentation. (Used for implementing LayerNorm and the CrossEntropy-Loss logic).